

TALLER 7

Bluetooth, HTTP GET, HTTP POST/JSON y adquisición analógica en tiempo real

Propósito. Esta guía está diseñada para que cada equipo avance de manera autónoma. En cada ejercicio se valida primero el hardware, luego el firmware del ESP32, después la comunicación y finalmente la app. No continúe a la siguiente etapa si la anterior no funciona.

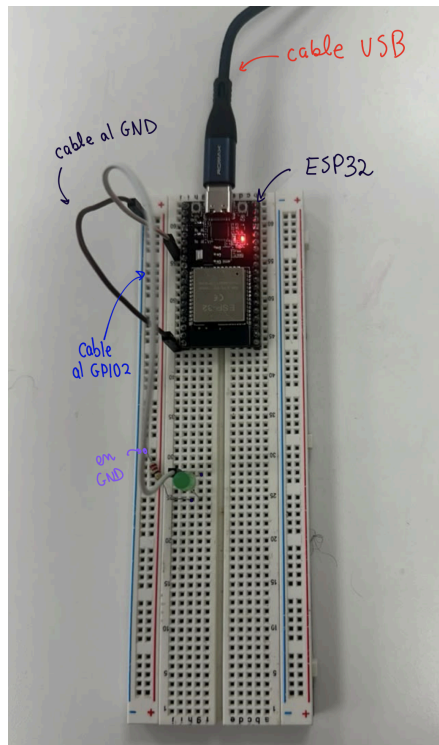
Regla de trabajo: Diagnostique por capas: 1) hardware, 2) código/ESP32, 3) comunicación, 4) MIT App Inventor. Cambiar varias cosas a la vez dificulta identificar el error.

Ejercicio 1 — Encender y apagar un LED por Bluetooth clásico

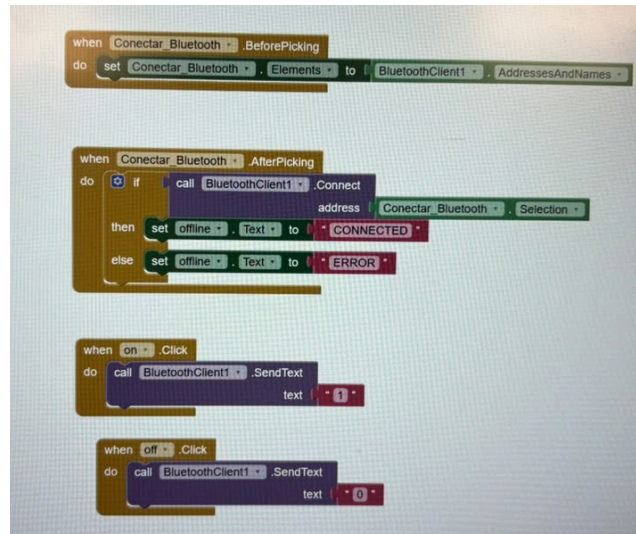
Objetivo. Controlar un LED conectado al ESP32 desde una app Android usando Bluetooth clásico. Este ejercicio introduce ListPicker, BluetoothClient, envío de texto y retroalimentación de conexión.

1.6 Evidencia a presentar

- ☐ 1 foto del circuito armado donde se identifique ESP32, LED y resistencia.



- ☐ 1 captura de pantalla de los bloques completos de App Inventor.



- ☐ Demostración en vivo: conectar por Bluetooth y ejecutar ON/OFF.
- ☐ Si el docente no puede observar en vivo: video corto (15–30 s) mostrando conexión y cambio del LED.

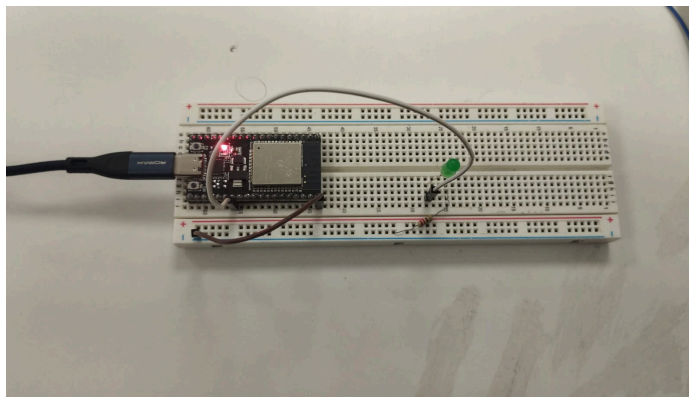
<https://drive.google.com/file/d/1qulpbBjvYnB6oBQleLlo2ZVi0TwSxztU/view?usp=sharing>

Ejercicio 2 — Control de dos LEDs por Wi-Fi usando HTTP GET

Objetivo. Crear un servidor web en el ESP32 y controlar dos salidas digitales mediante cuatro rutas HTTP GET.

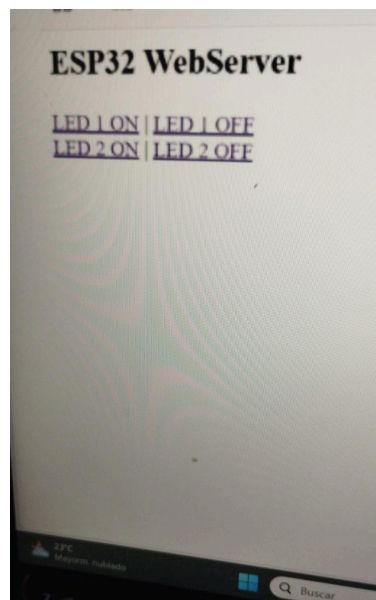
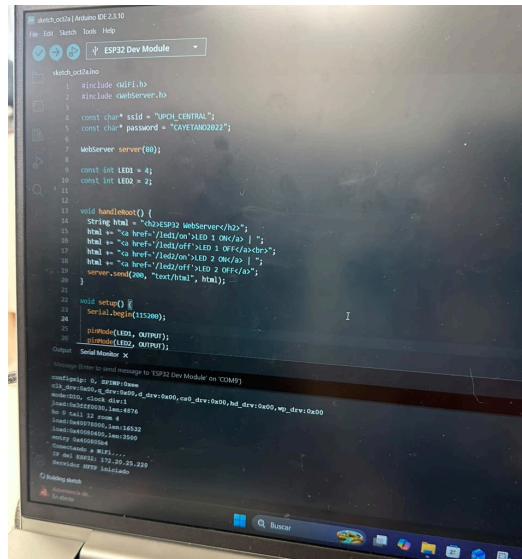
2.6 Evidencia a presentar

- ☐ Foto del circuito con los dos LEDs.



En el montaje se utilizó únicamente un LED externo porque la placa ESP32 ya cuenta con un LED integrado que puede ser controlado desde el programa. Por ello, se empleó el LED externo como una de las salidas digitales y el LED integrado de la placa como la segunda salida. Ambos pudieron ser encendidos y apagados desde la aplicación mediante las rutas HTTP GET definidas en el servidor web.

- ☐ Captura del navegador mostrando la página del ESP32.



- ☐ Captura de los cuatro bloques Click en App Inventor.

Screen1

ESP32 OUTPUTS controller

LED at GPIO4 is OFF

ON OFF

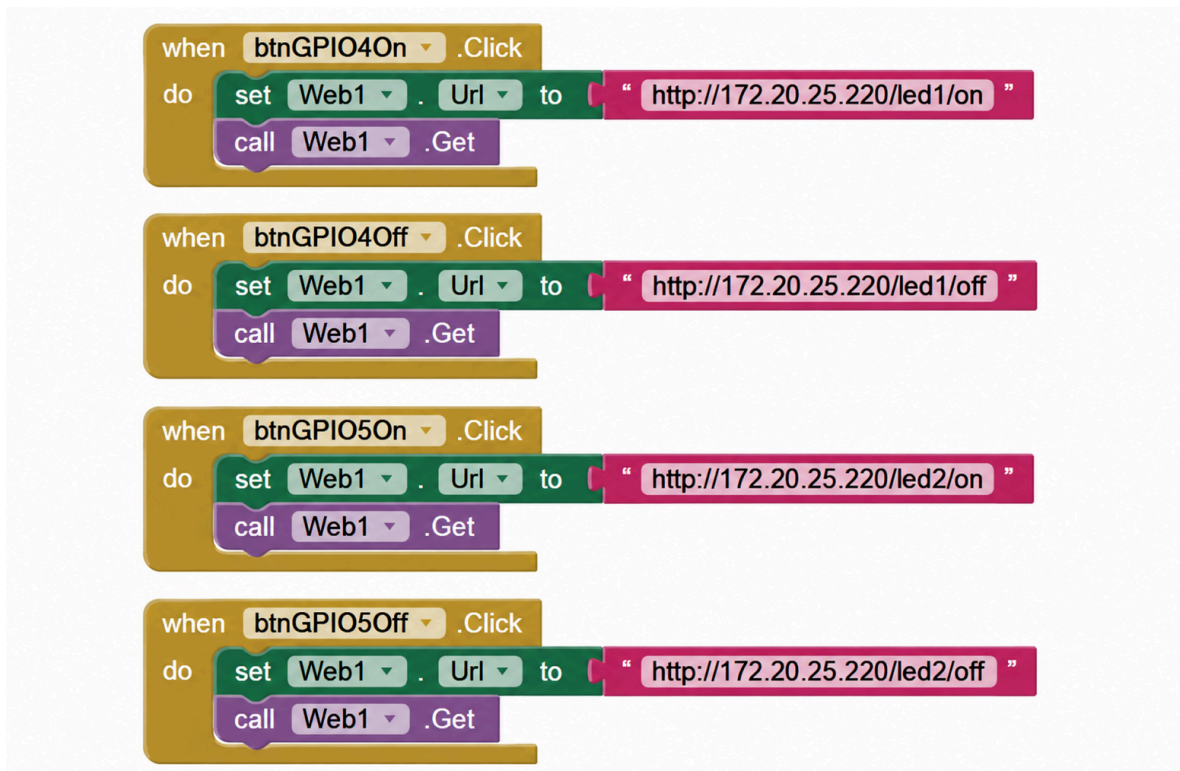
LED at GPIO5 is OFF

ON OFF

Current IP: http://172.20.25.220

Hint for TextBox1

SAVE



- ☐ Demostración en vivo: controlar ambos LEDs desde el celular.

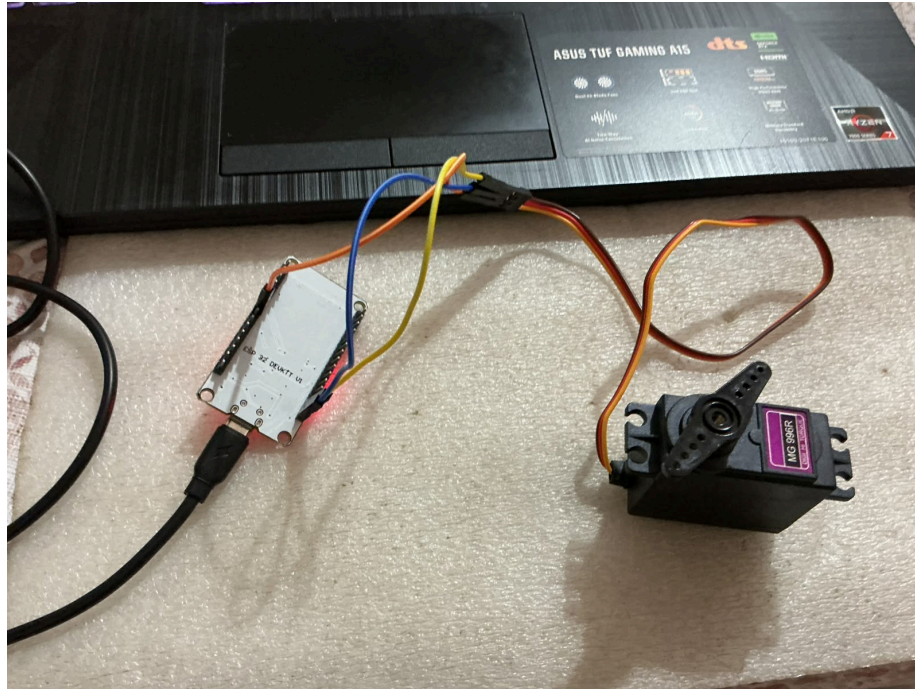
<https://drive.google.com/file/d/1-5FbsT27ZGqSTRbE-Mo7iaZFrhJyA3c/view?usp=sharing>

Ejercicio 3 — Control de un servomotor por Wi-Fi usando HTTP POST + JSON

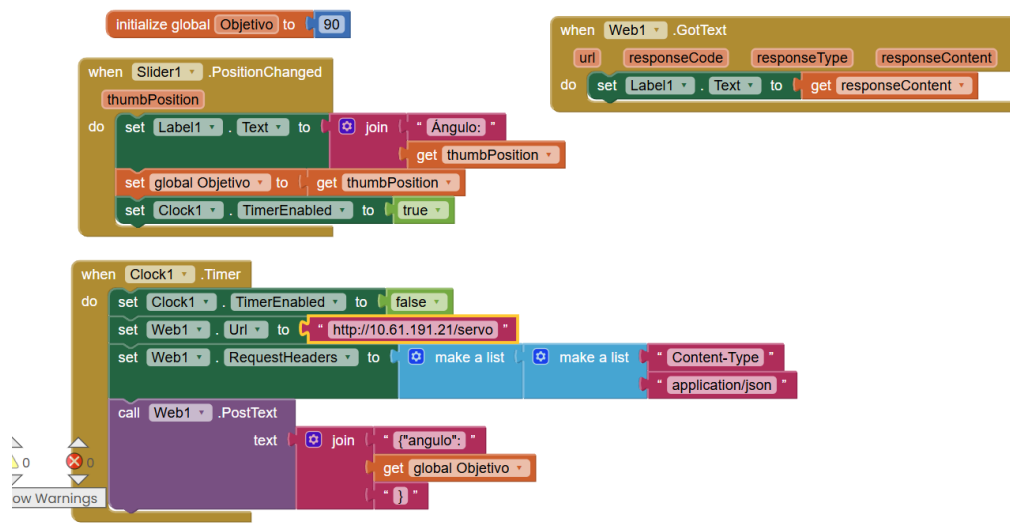
Objetivo. Enviar desde un Slider un valor de 0 a 180, empaquetarlo como JSON, recibirlo en el ESP32 y mover un servo.

3.9 Evidencia a presentar

- ☐ Foto del servo conectado al ESP32.



- ☐ Captura de todos los bloques de App Inventor.



- ☐ Captura del Monitor Serial mostrando un JSON recibido y el ángulo interpretado.

The screenshot shows the Arduino IDE interface. The top toolbar includes icons for checking, uploading, and running the sketch. The file menu shows 'sketch_oct4a.ino' is open. The code editor displays the following sketch:

```

63     delay(400);
64     Serial.print(".");
65   }
66
67   Serial.println();
68   Serial.println("WiFi conectado");
69
70   Serial.print("IP del ESP32: ");
71   Serial.println(WiFi.localIP());
72
73   server.on("/servo", HTTP_POST, handlePost);
74
75   server.begin();
76
77   Serial.println("Servidor HTTP iniciado");
78 }
79
80 void loop() {
81   server.handleClient();
82 }
83

```

The Serial Monitor at the bottom shows the output of the sketch:

```

-----
Angulo interpretado: 135
Respuesta enviada: {"ok":true,"angulo":135}
-----
Texto recibido: {"angulo":39.6}
Angulo interpretado: 39
Respuesta enviada: {"ok":true,"angulo":39}
-----
Texto recibido: {"angulo":163.8}
Angulo interpretado: 163
Respuesta enviada: {"ok":true,"angulo":163}

```

- ☐ Demostración en vivo o video corto moviendo el Slider y observando el servo.

<https://drive.google.com/file/d/1iHhywQc0bl1QIzlet7FJQvJFlwVrOOJG/view?usp=sharing>

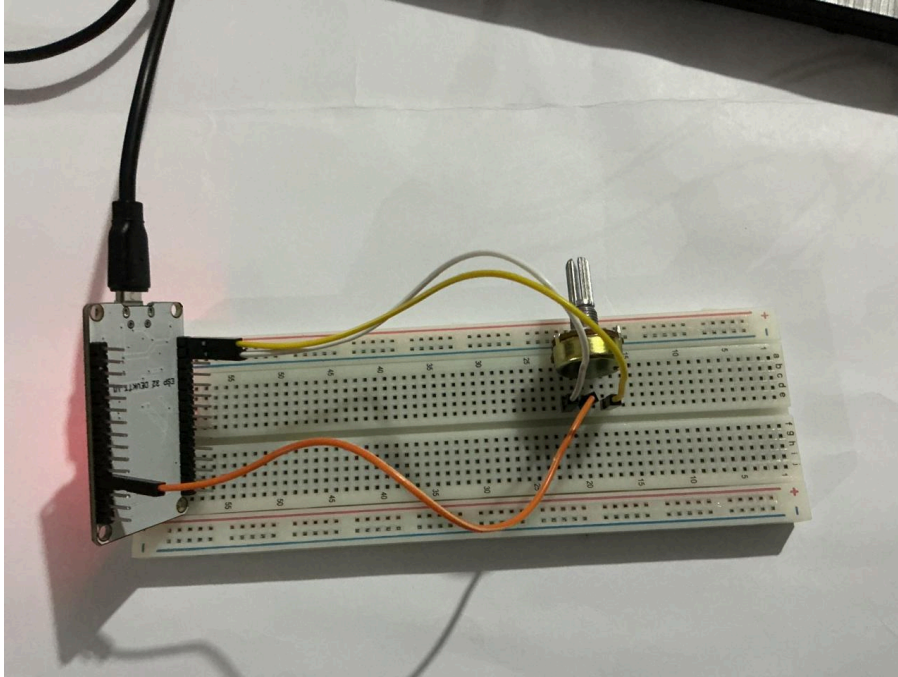
Ejercicio 4 — Adquisición analógica en tiempo real: potenciómetro → flex sensor

Estructura. Este ejercicio tiene dos etapas: 4A es una práctica guiada con potenciómetro para validar ADC + HTTP GET; 4B es la adaptación asíncrona calificable con flex sensor, calibración y visualización.

4A — Práctica guiada con potenciómetro

4A.6 Evidencia a presentar

- ☐ Foto del potenciómetro conectado.



- ☐ Captura del Monitor Serial mostrando variación del ADC.

```

1  const int PIN_ADC = 34;
2
3  void setup() {
4    Serial.begin(115200);
5  }
6
7  void loop() {
8    int valor = analogRead(PIN_ADC);
9
10   Serial.println(valor);
11
12   delay(200);
13 }
14
15

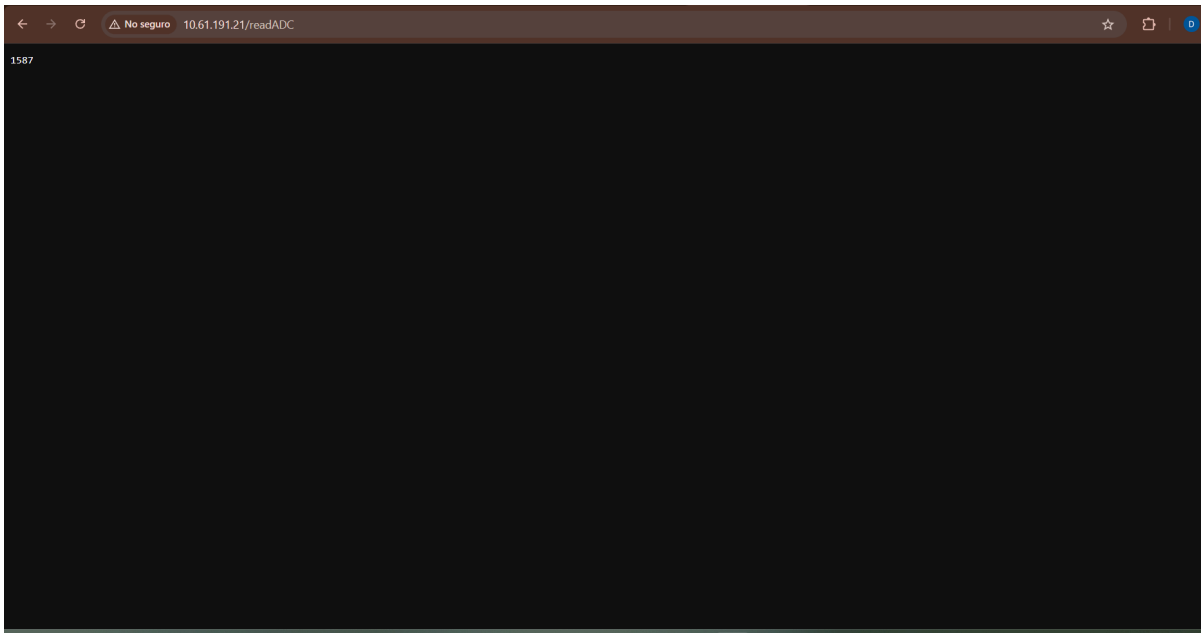
```

Output Serial Monitor X

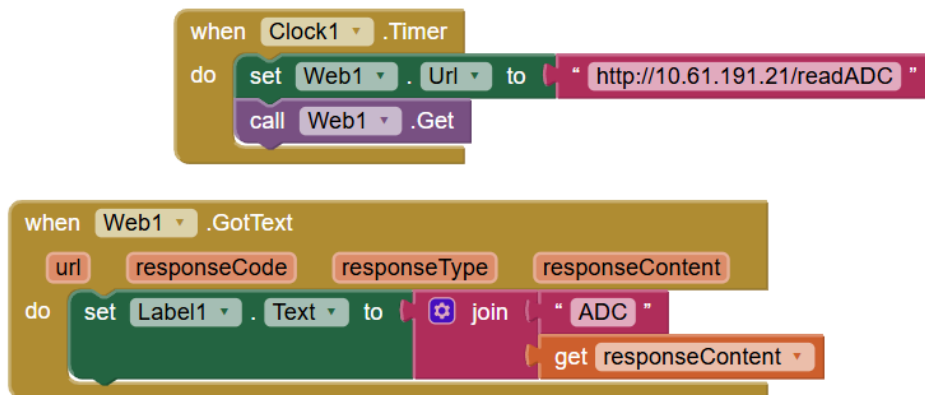
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')

3803
3759
3799
3787
3817
3794
3765
3799
3789
3793
3814

- ☐ Captura del navegador en /readADC.



- ☐ Captura de los bloques Clock1.Timer y Web1.GotText.



- ☐ Demostración breve: girar el potenciómetro y observar el valor en la app.

https://drive.google.com/file/d/1hEX-Wkwi91OwLQ-g6gypHIPvU_sT_26-/view?usp=sharing